

技术分享

智能体技术介绍 (上)



AI Agents

上篇 · 基础与原理：一个智能体是什么、又是怎么工作的

演进 · 原理 · 前沿 · 落地

王宇, 2026-06



本场地图 · 上篇

上篇打基础：本讲走四站，后两站下一讲见



1

为什么是现在

市场信号与一个直观对比



2

什么是智能体

定义、边界，以及何时‘不该’用



3

技术演进

规则系统 → LLM → 工具调用 → ReAct → 编排框架



4

工作原理

规划·记忆·工具·反思·循环 + 工程实践



5

↗ 下一讲 · 前沿与协作

多智能体 · MCP/A2A · 自进化 · 经济 · Computer Use



6

↗ 下一讲 · 现实与落地

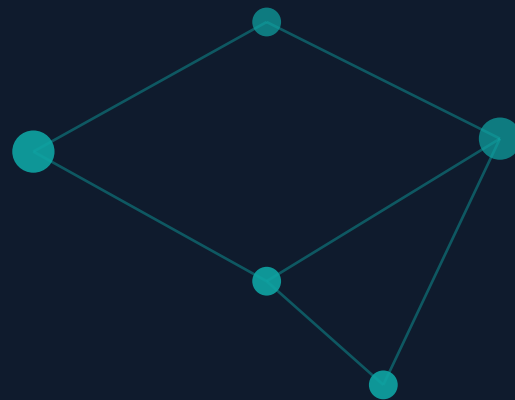
落地鸿沟 · 踩坑 · 评估 · 风控 · 决策树

01

WHY NOW

为什么是现在

智能体不是新词，但 2026 是它从‘能演示’走向‘进生产’的拐点。





市场信号

钱和落地都在往这个方向走

\$2019 亿

2026 年 agentic AI 预计支出

同比 +141% (Gartner)

40%

2026 年底内置任务型智能体的企业应用

2025 年还不到 5%

<25%

真正规模化进生产的比例

虽然 62% 已在试验 (McKinsey)



管理 takeaway: 热度真实，但同样真实的是——大量项目卡在‘试验’到‘规模化’之间。后面会讲为什么，以及怎么避坑。投入要选‘算得清 ROI、控得住风险’的场景。



一个直观对比

从‘会聊天’到‘会干活’

同一句话：「帮我订下周三去上海、下午到的直飞机票」

聊天机器人（给信息）

- 返回一段‘以下是一些航班...’的文字
- 你自己去比价、确认时间、下单
- 不知道你的日历、不记得你偏好直飞
- 航班满了？它不会知道，更不会换

= 信息 · 单步 · 被动

智能体（给结果）

- 查日历确认你那天有空
- 搜航班 → 按‘直飞 / 下午到’筛选比价
- 调用订票 API 下单（写操作前先跟你确认）
- 写进日历、发确认；满了就自动换备选

= 结果 · 多步自主 · 会适应

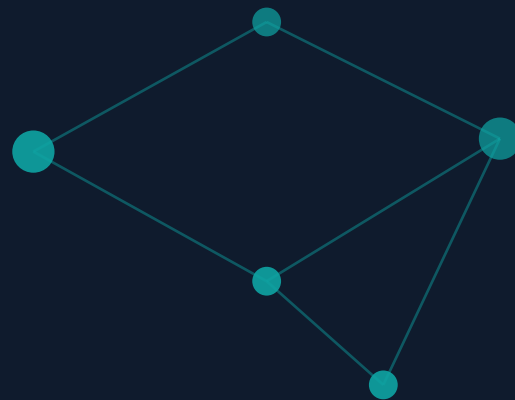
核心差异：能感知环境、自主规划多步、调用工具改变世界，并根据反馈调整——而不是一问一答。

02

WHAT IS AN AGENT

什么是智能体

先把概念厘清——它和聊天、RAG、固定工作流的边界在哪？





定义与边界

一个能自主‘干活’的闭环



是什么

- **感知** 读取输入、工具返回、环境状态
- **规划** 把目标拆成可执行的多步计划
- **行动** 调用工具/API, 真正改变世界
- **反思** 看结果、纠错、再决定下一步

不是什么

- ≠ 聊天机器人: 只产文字、不行动、不闭环
- ≠ 纯 RAG: 只是把检索到的资料塞进上下文
- ≠ 固定 workflow: 步骤由人写死, 不自主决策



四个层级

‘智能体’这个词，别人用得松

层级	典型形态	谁在做决策	适应性 / 自主性
L0 单次问答	ChatGPT 直接回答	人	无
L1 检索增强 RAG	知识库问答、客服 FAQ	人	无，仅补充上下文
L2 固定 workflow	预设的提示链 / 编排	开发者预先写死	低，照脚本走
L3 智能体	自主完成开放式任务	模型自主、动态规划	高，按中间结果调整

判断标准：自主性 + 适应性。前三层本质是‘人/开发者在决策’，只有 L3 把决策权交给了模型——这也是它更强、但更贵更难控的根源。



反直觉但重要

什么时候，你其实‘不需要’智能体



一条红线：如果你的‘智能体’对相同输入总是产生相同输出——那它只是一个工具，不是智能体（用确定性代码就好）。

上 Agent 的代价（常被低估）

- 更贵：一个任务往往是多次 LLM 调用
- 更慢：多轮推理 + 工具往返
- 更难调试：每次轨迹都可能不同
- 不确定：同样输入不保证同样结果

什么时候才值得上

- 步骤依赖中间结果、无法预先枚举
- 需要在多个可能路径里‘判断’怎么走
- 任务开放、长程，规则写不完
- 容错较高、出错代价可控的场景

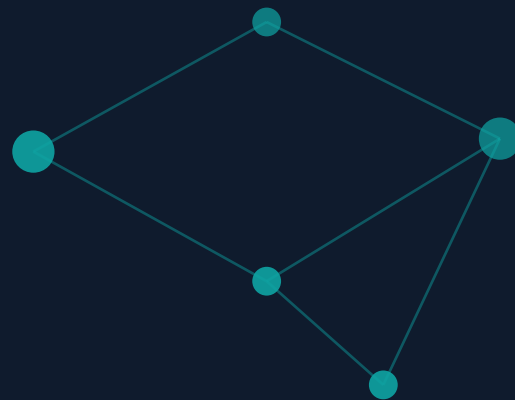
经验法则：先用最笨、最确定的方案能解决，就别上 agent。能枚举的流程 → 工作流/代码；真要判断 → 才上 agent。

03

EVOLUTION

技术演进

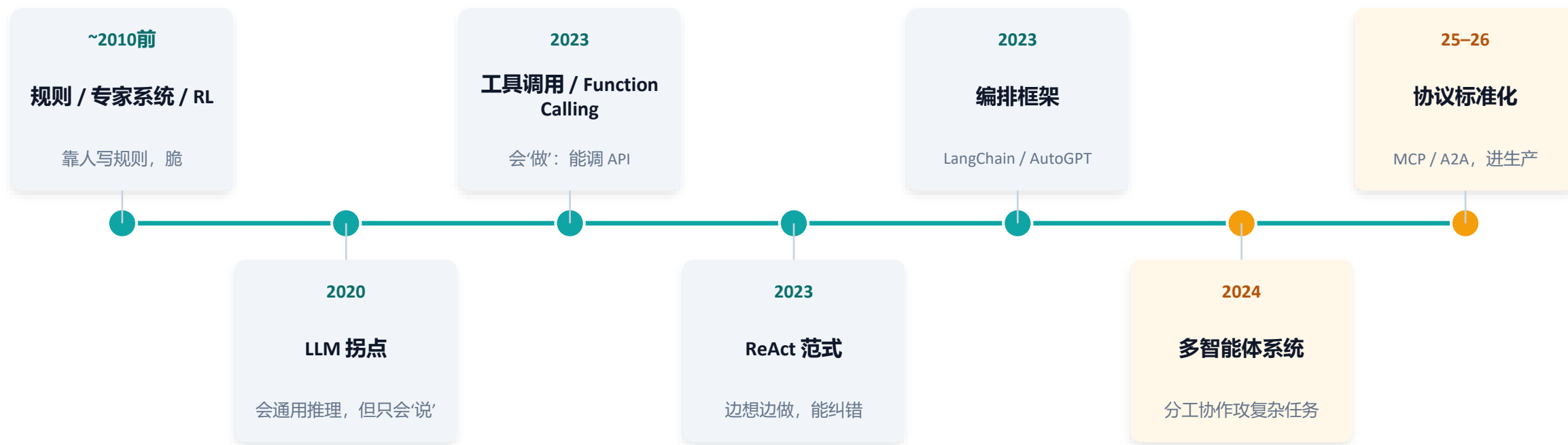
每一步，都是在解决上一步的局限——从‘会说’一路走到‘会自主干活’。



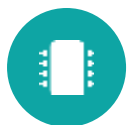


一条主线

智能体能力的七级跳

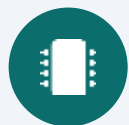


下面逐站展开，并配上真实的提示词、调用格式与协议消息。



起点

早期智能体：聪明，但脆



规则 / 专家系统

if-then 规则库（如医疗诊断 MYCIN）。规则一多就互相打架、难维护。



强化学习智能体

靠奖励试错学策略（如 AlphaGo）。强在闭环游戏，难迁移到开放任务。

共同的天花板：智能来自人手写的规则或狭窄环境的训练，**没有通用的语言理解与常识**。给个没见过的新任务，就抓瞎。这正是大模型要解决的。



两步关键跃迁

LLM 让它会‘说’，工具调用让它会‘做’

大模型带来通用推理与语言理解；但要真正‘干活’，得能调用外部工具——这就是 Function Calling。

TOOL SCHEMA

```
// 工具定义：给模型看的“说明书”
{
  "name": "get_weather",
  "description": "查询城市实时天气",
  "input_schema": {
    "type": "object",
    "properties": {
      "city": {"type": "string"},
      "unit": {"enum": ["c", "f"]}
    },
    "required": ["city"]
  }
}
```

MODEL OUTPUT

```
// 模型不直接答，而是“决定调用”
{
  "type": "tool_use",
  "name": "get_weather",
  "input": { "city": "上海" }
}
```

洞见：那段 description 就是‘提示词’——写得越清楚，模型用得越准。schema 是模型与世界之间的契约。

运行机制：模型输出‘要调哪个工具+参数’ → 你的代码真正执行 → 把结果回填给模型 → 模型继续。



ReAct: 把‘想’和‘做’交织起来

PROMPT 模板

你可以使用这些工具: {tool_descriptions}
按下面格式循环, 直到能回答:

Thought: 我现在该做什么 / 为什么

Action: 要调用的工具名

Action Input: 工具参数 (JSON)

Observation: (系统回填工具结果)

... (Thought/Action/Observation 可重复 N 次)

Thought: 我已经知道答案了

Final Answer: 给用户的最终回答

一次真实 TRACE

问题: 现在北京和上海哪个更热?

Thought: 需要两地气温, 先查北京

Action: get_weather

Action Input: {"city": "北京"}

Observation: 北京 19°C

Thought: 再查上海

Action: get_weather

Action Input: {"city": "上海"}

Observation: 上海 24°C

Thought: 24>19, 上海更热

Final Answer: 上海更热 (24 vs 19°C)

为什么有效: 把推理显式写出来, 模型就能‘边想边做’——上一步错了, 下一步能在 Thought 里发现并纠正。这就是从单次问答到多步自主的关键。



工程化第一波

编排框架时代 (2023)

LangChain

把‘提示+工具+记忆+链’封装成组件，快速搭原型。

生态大、上手快

AutoGPT

让 agent 自主拆目标、循环执行，引爆了想象力。

炫，但常跑飞、烧钱

LangGraph / 状态图

用图/状态机显式控制流程，可回退、可断点。

更适合生产

踩坑经验：框架让 demo 飞快，但生产里‘可控性’才是命门。早期 AutoGPT 类的全自主循环很容易跑飞、死循环、烧 token——这也是后来‘确定性护栏’与状态图编排兴起的原因。



工程化 · 选型

主流开源智能体框架对比

没有银弹——按 **上手难度 · 控制力 · 生产成熟度 · 团队栈** 选型；下表是 2026 年中的主流选择。

框架	范式 / 一句话定位	上手	生产成熟度	最适合
LangGraph	状态图：节点 + 共享状态	陡	最成熟 · 检查点/HITL/可观测	长程、可断点续跑、需人审的生产系统
CrewAI	角色制 crew，像团队分工	易	中等 · 生态成长中	30 分钟搭多智能体原型；流程能映射到角色
AutoGen / AG2	对话式多智能体（群聊/辩论）	中	转维护，微软主推 Agent Framework	多方对话、共识、人审型任务
OpenAI Agents SDK	轻量，handoff 交接（取代 Swarm）	低	高 · 内置 tracing/guardrails	OpenAI 原生、单/少 agent 快速起步
Google ADK	分层 agent 树（根 → 子 agent）	中	早期 · 绑 Vertex AI/Gemini	层级委派、Google Cloud 技术栈
Claude Agent SDK	原生 MCP · 上下文隔离子 agent	低-中	高 · 安全优先	MCP 生态、子 agent 隔离、安全敏感

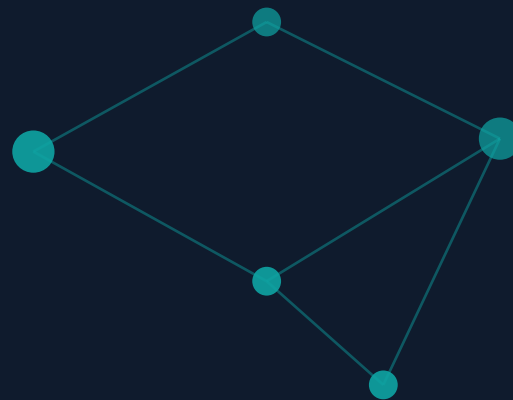
选型口诀：先用最小 SDK 把**单 agent**做好；要状态/可断点/HITL → **LangGraph**；要快速角色分工 → **CrewAI**。多数生产系统是**混用 + MCP/A2A 互联**。人气参考：Dify 星标最高、LangGraph 企业下载量最大。

04

HOW IT WORKS

工作原理

拆开一个智能体：规划、记忆、工具、反思、循环——以及怎么‘调教’它。





解剖一个智能体

五大要件 + 一个执行循环



- **规划 Planning**: 把目标拆成子步骤, 决定先做什么
- **记忆 Memory**: 短期(上下文) + 长期(向量库), 记住事实与偏好
- **工具 Tools**: 函数/API/检索/代码执行, 是它的‘手’
- **反思 Reflection**: 检查结果、发现错误、调整策略
- **执行循环**: 把以上串成一个‘做—看—再做’的闭环, 直到完成或触发终止条件



先想清楚怎么干，再动手

CoT 思维链

‘想清楚再答’，把推理写出来

Plan-and-Execute

先出完整计划，再逐步执行，可审查

ReAct

边想边做，适合需要试探的任务

Tree-of-Thoughts

多路径探索，择优而行

示例：任务「做 Q3 销售季报」被分解为——

① 取数 → ② 清洗 → ③ 按区域汇总 → ④ 算同比/环比 → ⑤ 生成图表 → ⑥ 写摘要 → ⑦ 导出 PDF

经验：显式规划 = 可控 + 可中途人工审查。长程任务尤其要先出计划，否则 agent 容易走着走着忘了目标。



ReAct: 边想边做, 错了能当场改

核心: 想一步·做一步·看一步

- **Thought**→**Action**→**Observation** 三段循环, 直到给出最终答案
- 每一步都基于上一步的真实反馈, 再决定下一步怎么走
- 适合信息不全、要边探边定的任务 (查库、多跳问答、排障)
- **代价**: 步数多、易绕圈——必须设步数上限与终止条件

一次真实轨迹 · 退货咨询

问题: 上周买的鞋能退吗?

Thought: 先查这笔订单

Action: `query_order("最近")`

Observation: 鞋 6/3 已签收

Thought: 看是否在 7 天无理由内

Action: `get_policy("无理由退")`

Observation: 签收后 7 天可退

Thought: 6/3 到今天 5 天, 可退

Final Answer: 可退, 剩 2 天,
要我帮你发起吗?

ReAct ≠ 先把全程想好再做。它是‘想一步→做一步→看一步’, 所以能在中途发现错误并纠正; 也因此最容易兜圈子——记得加步数上限与终止条件。



Plan-and-Execute: 先列计划, 再照单执行

两段式: 先规划, 后执行

- **Planner**: 先把目标拆成一份完整的步骤清单
- **Executor**: 拿到清单后照单逐步执行
- 计划可先交人审查 / 修改, 再开跑——可控性强
- **对比 ReAct**: P&E 先想全程再走, 省调用、但应变较差

示例 · 订一张机票

```
// ① PLAN 一次性产出
1. 查日历, 确认 6/18 有空
2. 搜 沪→京 6/18 下午到 直飞
3. 按价格+到达时间排序选最优
4. 调订票工具下单 (需确认)
5. 写日历 + 发确认邮件

// ② EXECUTE 照计划逐条做
step1 ✓ 当天空闲
step2 ✓ 命中 6 个航班
step3 ✓ 选 MU5102 ¥1280
step4 || 暂停, 等你确认下单
```

计划与执行分离 = 先对齐、再动手。计划阶段就能人工审查, 避免‘走错一里地’; 若中途环境变了 (如航班售罄), 通常要 Re-plan 重新规划。



Tree-of-Thoughts: 多想几条路，择优而行



怎么运作：把思考变成一棵树

- 每一步生成多个候选‘想法’，分头展开，而非只走一条
- 用打分 / 投票评估各分支，剪掉差的，沿最优继续（像下棋搜索）
- 适合解空间大、一条道走到黑容易错的难题
- 代价：最贵——分支越多，LLM 调用成倍增长

例·解‘24 点’：同时试 先加 / 先乘 / 先减 三条路，评估哪条最可能凑出 24，只保留最优分支继续。

ToT = 把‘链式思考’升级为‘树式搜索’。用算力换正确率——只在高价值、容错低、值得多花钱的难题上用；日常任务用 ReAct 往往更划算。



它记得多少，决定它能做多复杂



短期记忆

上下文窗口里的对话与中间结果。会满、会贵、长了会‘忘中间’。



长期记忆

向量库/数据库，跨会话记住事实与偏好，用时检索回来(RAG)。



工作记忆

当前任务的 scratchpad / 待办清单，维持目标不跑偏。

关键经验（这就是‘上下文工程’）：不是把所有东西都塞进上下文，而是‘在对的时间，给对的信息’——靠摘要压缩历史、按需检索记忆、裁剪无关内容。塞满窗口 = 又贵、又慢、还更容易出错。



短期记忆：这一轮看得见的上下文

短期记忆 = 模型这一轮能直接「看见」的全部内容，也就是上下文窗口。即取即用、零检索延迟，但会满、会贵、会忘。

里面装着什么

- **配置类 configs**：人设、能力、约束、工具说明（system prompt 那部分）
- **近期事件**：用户消息、点击、工具返回、系统提示
- **当前工作区**：本轮对话 + 中间结果（草稿、检索片段）

特点与三个坑

- **快**：即取即用，0 检索延迟
- **会失忆**：会话一结束就清空，不跨会话保留
- **会满会贵**：内容越堆越多，token 成本线性上涨
- **中间易丢**：lost-in-the-middle / 上下文腐烂——太长就漏掉中段

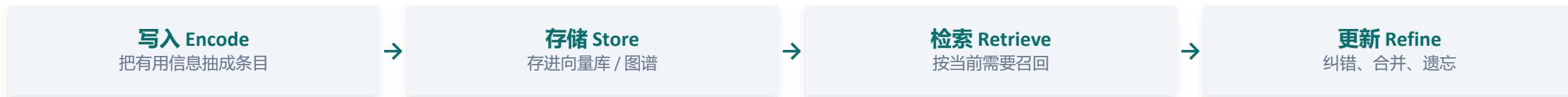
一句话：短期记忆 ~ 摊在工作台上的纸——不是越大越好。 **管理手段**：滚动摘要、按需裁剪、只留有用片段；怎么往这张「纸」上放对内容、放多少，正是「上下文工程」要解决的核心问题。



记忆 · ② 长期记忆

长期记忆：跨会话的知识库（≈ RAG）

长期记忆 = 跨会话持久保存的知识，用时只检索一小段注入上下文——本质是给 agent 配了一套 RAG。



存储形态

- **向量库**：把内容变 embedding，按「语义相近」召回
- **知识图谱**：实体 + 关系，支持结构化查询
- **2026 主流**：Hybrid 混合——向量 + 图谱 + 元数据过滤

检索与服务化

- **混合检索**：语义相似 + 关键词 + 实体匹配，取最相关几段拼进 prompt
- **Memory-as-a-Service**：Mem0 / Zep / Letta 把记忆从推理循环里解耦出来

两个实战提醒：① **要会「遗忘」**——过期/错误的记忆比没有更糟（上下文腐烂）；② **算笔经济账**——小规模/单用户场景，直接上百万 token 长上下文，可能比维护一整套向量库还省。



工作记忆：盯住当前任务的草稿纸

工作记忆 = 为「当前这个任务」临时维护的草稿区：**计划、待办、中间结果**——专门用来盯住目标、不跑偏。

是什么 · 和短期记忆的区别

- **scratchpad**：计划、TODO、中间结果、当前进度
- **作用**：长程任务里维持目标，每步回看、不偏航
- **vs 短期记忆**：是被「主动维护」的任务状态，不是原始对话流
- **Letta 的做法**：借鉴操作系统，在即时上下文与长期存储间「分页」搬运

AGENT 的工作记忆 (scratchpad)

任务：生成 Q3 季度报告

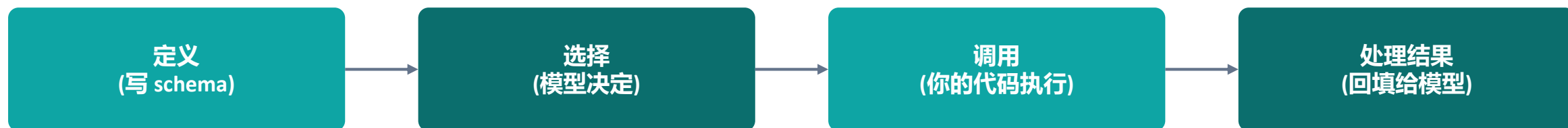
- [✓] 1. 取数 (库 → 原始表)
- [✓] 2. 清洗 / 对齐口径
- [] 3. 汇总指标 ← 当前
- [] 4. 生成图表
- [] 5. 写执行摘要

打勾 + 标「← 当前」，让 agent
始终知道下一步该做啥

三种记忆各司其职：短期 = 这一轮对话的上下文 · 长期 = 跨会话的知识库 · 工作 = 盯当前目标的草稿纸。三者协同，agent 才能既「记得久」又「不跑偏」。



工具是它的‘手’——也是最大风险面



常见坑

- 工具太多(>15~20): 模型会选错或乱试
- description 含糊: 模型不知道何时该用
- 返回结果太长: 撑爆上下文、烧 token

应对经验

- 工具分组 / 按场景按需加载
- schema 描述写清‘何时用、参数含义’
- 结果做摘要或分页, 只回填关键字段

安全红线: 工具= 真实副作用 (删库、转账、发邮件)。写操作必须配权限控制+ 人工确认。



会自我纠错，正确率才上得去

自我批判 Self-critique

让模型回看自己的输出，主动找错并改

Reflexion

失败后总结‘教训’写进记忆，下次不再犯

外部校验 Verifier

用规则/单测/另一个模型验证（如代码跑测试）

洞见：反思能显著提升正确率，但每次反思都是额外的 LLM 调用（更贵更慢）。对‘高风险、可校验’的任务最划算——比如写代码可以跑测试、出报告可以核对数字。

REFLECTION PROMPT

```
// 一个简单的反思提示词
检查上面这段 SQL：
1) 有语法错误吗？
2) 是否答非所问、漏了过滤条件？
3) 若有问题，直接输出
   修正后的版本。

否则输出："通过"
```



把五件套跑起来

一次完整的执行循环（走一遍）

任务：「查上季度华东区销售额，做成柱状图，发到我邮箱」

#	思考(Thought)	动作(Action)	观察(Observation)	类型
1	先要拿到数据	<code>sql_query(...region='华东'...)</code>	返回 6 行销售数据	工具调用
2	数据有了，画图	<code>plot_bar(data)</code>	chart.png 已生成	工具调用
3	该发邮件了	<code>send_email(to=我, 附件=chart.png)</code>	暂停·等待你确认	需人工确认
4	执行发送	—	已发送✓	完成

注意第 3 步：发邮件是‘写操作/有副作用’，agent 在这里停下来等人确认——这就是“人在回环”（human-in-the-loop）。



从‘提示词工程’到‘上下文工程’

提示词工程 (Prompt Eng.)

- 关注‘怎么问这一句’
- 措辞、角色设定、few-shot 示例
- 面向单次交互
- 像写好一道题目

上下文工程 (Context Eng.)

- 关注‘窗口里到底放什么’
- 动态组装：系统指令+工具+检索记忆+历史摘要+当前状态
- 面向多步、长程的系统供给
- 像设计一套‘信息供给系统’

一句话：提示词是‘一句话’，上下文是‘一套系统’。2026 业界共识——让 agent 在生产中可靠，胜负手在上下文工程，而非又换一个更大的模型。



一个生产级 System Prompt 的骨架

SYSTEM PROMPT

角色

你是企业数据分析助手，严谨、谨慎。

目标

用提供的工具查数据，给出有依据的结论。

可用工具

- `sql_query(q)` 只读查询数仓
- `plot(spec)` 生成图表

约束（硬规则）

- 任何写/发送操作，必须先向用户确认
- 不确定就说不确定，禁止编造数字
- 金额必须标注口径与时间范围

输出格式

先结论 → 再依据（用了哪些数据） → 下一步建议

示例

{ 放 1~2 个 few-shot 示范 }

为什么这样写

- **角色+目标**：先框定身份与边界
- **工具清单**：模型才知道有哪些‘手’
- **硬约束**：把安全与口径写死——生产可靠性的命门
- **输出格式**：结构固定，便于下游解析与人审
- **few-shot**：给范例比讲道理更管用



训练数据长什么样：① 函数调用（Function Calling）

教模型「该调工具时就调、拿到结果接着答」。一条训练样本，就是一段带 tool_calls 的多轮对话（messages 格式）：

一条 function-calling 训练样本

```
[
  // 1 系统声明可用工具
  {"role": "system",
   "content": "可调用 get_weather(city)"},

  // 2 用户提问
  {"role": "user",
   "content": "北京今天要带伞吗? "},

  // 3 模型决定调用工具 (content 为空)
  {"role": "assistant", "content": null,
   "tool_calls": [{"id": "c1",
                    "name": "get_weather",
                    "args": {"city": "北京"}}]},

  // 4 工具结果回填 (用 id 配对)
  {"role": "tool", "tool_call_id": "c1",
   "content": "{rain:true, mm:6}"},

  // 5 模型据结果给人话答案
  {"role": "assistant",
   "content": "会下雨, 记得带伞 ☂️"}
]
```

怎么读这段数据

- **system**: 声明有哪些工具、怎么用
- **user**: 真实问题
- **assistant + tool_calls**: 模型「决定」调用, 此时 content = null
- **tool**: 把真实结果回填, 用 id 和上一步配对
- **末尾 assistant**: 根据结果给出人话答案

关键不是「问 → 答」配对, 而是教会成对的「**决定调用 → 结果回填 → 据此续答**」这一多轮工具循环——让模型学会「什么时候该停下来调工具」。



训练数据长什么样：② 轨迹数据 (Trajectory)

函数调用教「单步怎么调工具」；轨迹数据教「整件事怎么一步步做完」——记录完整的「想-做-看」序列，并标注成败和奖励。

一条 trajectory 训练样本

```
{
  "task": "把发票金额录入系统",
  "steps": [
    // 第 1 步: 想 → 做 → 看
    { "thought": "先读出 PDF 金额",
      "action": "open_pdf",
      "input": { "file": "inv_07.pdf" },
      "observation": "金额 = 1280" },

    // 第 2 步
    { "thought": "再填进 ERP 表单",
      "action": "type",
      "input": { "amount": "1280" },
      "observation": "提交成功" }
  ],
  "success": true,
  "reward": 1.0    // 任务级奖励
}
```

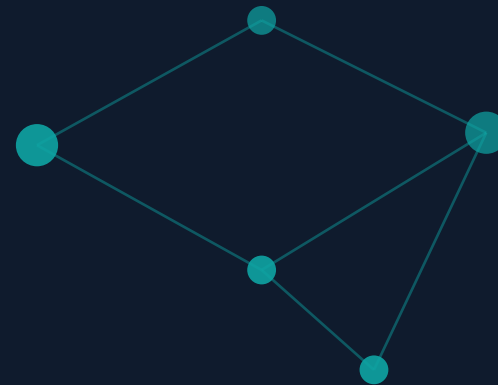
为什么要整条轨迹

- **一条 = 一次完整任务**：「想 → 做 → 看」的序列 + 最终成败/奖励
- **用途**：训练规划能力 + 强化学习 (RL / RFT)，从成功轨迹里学策略
- **来源**：真实运行日志 / 合成生成 / 人工修正
- **奖励信号**：可任务成功 1/0，也可逐步打分

两类数据，两种教法：① 函数调用教「工具怎么调、结果怎么接」；② 轨迹 + 奖励教「整体怎么规划」。**后者正是下一节『自进化』的燃料**——agent 从自己的成功轨迹里持续学习。

带走这五句

上篇小结



- 1 智能体 = 会自主干活的大模型 — 规划 + 记忆 + 工具 + 反思 + 执行循环
- 2 不是所有事都该上 agent — 能用确定性脚本 / RAG 解决的，别硬上；先算清 ROI
- 3 三种记忆各司其职 — 短期=本轮上下文 · 长期=跨会话知识库 · 工作=当前任务草稿纸
- 4 ReAct 是地基 — 想 → 做 → 看 的循环；Plan-Execute / ToT 都是它的变体
- 5 训练数据决定能力上限 — 函数调用教「单步调工具」，轨迹 + 奖励教「整体规划」

下一讲（下篇）：多智能体协作 · 协议(MCP/A2A) · 自进化 · 智能体经济 · Computer Use · 工程落地。 我们下次见。